VPN-LAN Service Access — Comprehensive Phased Plan

Revision: 1 **Last modified:** 2026-07-01T16:05:00Z **Status:** Planning (Phase-0 scaffold next) — feature workstream on branch `feature/vpn-aware-dynamic-routing` (§11.4.167) **Authority:** Inherits `constitution/Constitution.md` per §11.4.35. §11.4.172 phased-planning doc + §11.4.167 big-work-item feature-workstream lifecycle for the VPN-LAN service-access feature. **Companion:** summary `PLAN_Summary.md` · integration status `Status.md` (created when ≥2 phases land)

---

1. Goal (operator mandate, verbatim intent)

Make **every mainstream service exposed on a VPN-internal network reachable and usable through helix_proxy**, driven by the sibling `svord_toolkit` VPN bridge, and prove each one works with **real captured evidence, no false results, no bluff** (§11.4 / §11.4.69 / §11.4.107). Protocols in scope (operator-enumerated across four messages):

- **File shares:** SMB / CIFS / NMB (NetBIOS), NFS
- **File transfer:** FTP / FTPS, SFTP, WebDAV
- **Email:** IMAP / IMAPS, SMTP / submission, POP3 / POP3S
- **Discovery:** mDNS / DNS-SD (Bonjour), SSDP / UPnP, WS-Discovery, DIAL
- **Casting:** Google Chromecast, Miracast
- **Device bridge:** ADB (Android Debug Bridge) — access, debug, connect, **flash**
- Everything mainstream; full documentation; full test coverage; containerized via the containers submodule (§11.4.76).

Hard constraints (operator, in force):

1. Use the sibling `svord_toolkit` project's `svord-ssh-*` script(s) to connect to the remote network — but the connection-script name MUST be **parametrized via an environment variable** (decoupled §11.4.28), **fully documented**, **NOT git-versioned**, with a **`.env.example`** illustrating the bridge (in our case pointing at `svord_toolkit`'s `svord-ssh-*` script).

2. **Do NOT change anything** in `svord_toolkit` / `svord_factory` or on any remote connected host without an **interactive question with options** first (§11.4.66 / §11.4.122).
3. Everything covered by **all supported test types** (§11.4.169) + **Challenges** (challenges submodule) + **full HelixQA banks + autonomous HelixQA sessions** (§11.4.27).
4. Commit + push all submodules + main repo **regularly to all upstreams** (§2.1 / §11.4.113 no-force).
5. Report when **fully complete**.

---

2. Architecture — the one-line routing map (recon-derived, FACT/§11.4.6)

The VPN is **L3-routed** (WireGuard + L2TP/PPP over `ppp0`; reachable subnet `10.0.0.0/8`; svord host `10.6.100.221`). This single fact drives every protocol decision. SOCKS5/Squid is the WRONG primitive for mounts and discovery; an **L3 routed gateway + scoped SSRF allowlist** is correct.

Traffic class	Correct primitive	Protocols
Unicast TCP	**ROUTE** over the L3 VPN (HTTP-shaped also works through existing **Squid**)	SMB/445, NFS/2049, FTP-control/21, SFTP/22, IMAP/993, SMTP/587, POP3/995, WebDAV (HTTP), Cast control 8008/8009, DIAL HTTP
Unicast UDP (peer IP known)	**ROUTE** over the L3 VPN	NFS aux, NMB/137-138 (unicast fallback)
FTP data channel	**ROUTE** the server's pinned passive-port range (single caveat)	FTP/FTPS passive
Multicast **discovery**	**REFLECT** on the remote (device-side) network — routers do not forward it across L3	mDNS 5353, SSDP/UPnP 1900, WS-Discovery, DNS-SD, Chromecast/DIAL discovery
L2 / Wi-Fi-Direct	**STRUCTURALLY-IMPOSSIBLE** over L3 VPN (§11.4.112)	**Miracast**

Rule of thumb: unicast services route; HTTP-shaped services already work through Squid; multicast discovery needs a remote-side reflector; Wi-Fi-Direct/L2 is out of scope by physics. Deep-research citations recorded in §11 (per §11.4.150).

---

3. The env-var bridge contract (§11.4.28 decoupled, NOT git-versioned)

helix_proxy MUST NOT hardcode `svord_toolkit` anywhere. It reads a **bridge contract** from the environment (real values live in a gitignored `.env`; a tracked `.env.example` documents the shape). The bridge is 4 operator-supplied hooks + a working dir:

Env var	Meaning	Example value (in `.env.example`, illustrative only)
`HELIX_SVORD_DIR`	Path to the sibling bridge project	`../svord_toolkit`
`HELIX_BRIDGE_CONNECT`	Command that brings the VPN up	`${HELIX_SVORD_DIR}/svord-ssh-connect`
`HELIX_BRIDGE_DISCONNECT`	Command that tears the VPN down	`${HELIX_SVORD_DIR}/svord-ssh-disconnect`
`HELIX_BRIDGE_HEALTH`	Command/probe that reports reachability (exit 0 = up)	`${HELIX_SVORD_DIR}/svord-ssh-health`
`HELIX_BRIDGE_SUBNET`	The reachable remote subnet (for route + SSRF allowlist scoping)	`10.0.0.0/8`
`HELIX_BRIDGE_HOST`	A known remote host for smoke reachability	`10.6.100.221`

Rules: (a) `.env` is gitignored (§11.4.30) and never contains secrets in-tree (§11.4.10 — names/paths only); (b) `.env.example` is tracked and is the §11.4.77 re-obtain mechanism (documents how to point the bridge at `svord_toolkit`); (c) helix_proxy invokes only these hooks — it never reaches into `svord_toolkit` internals and never modifies it (§11.4.122); (d) a `svord-doctor` preflight resolves +

validates the bridge and, when the bridge is down, every downstream test **honestly SKIPs** (`network_unreachable_external` per §11.4.3 / §11.4.69) — never a fake PASS.

---

4. Security reconciliation (must land BEFORE any egress widening)

Widening egress to the VPN subnet reopens SSRF surface. Reconcile per §11.4.120 (fix-breaks-its-own-gate) + OWASP SSRF Case-1, keeping the existing hardening intact:

1. **Keep the RFC1918 / link-local / loopback / metadata block as the floor** (the `4626f05` Dante SSRF + Squid ACL work stays). Do NOT remove it.
2. **Add a narrow allowlist carve-out** for `HELIX_BRIDGE_SUBNET` only (Dante first-match top-down: the factory-subnet ALLOW rule sits ABOVE the broad internal-DENY; every other RFC1918 range stays denied). The S1/S3/S4 guards (now GREEN + wired §11.4.135) must stay GREEN after the carve-out — the S1 RED teeth prove a still-blocked target still denies.
3. **Open-relay guard (email, §4-recon FACT):** never expose anonymous CONNECT-to-:25. Route authenticated **submission** (587/465) to VPN clients; keep :25 server-to-server behind the boundary. The SSRF allowlist must NOT make helix_proxy an anonymizing spam conduit.
4. **STARTTLS-stripping caveat:** prefer routing implicit-TLS ports (993/995/465, RFC 8314) over plaintext-upgradable STARTTLS ports where the choice exists.
5. Every widening ships with a paired §1.1 mutation proving the allowlist has teeth (an out-of-allowlist target still denies).

---

5. Phase breakdown (tasks → subtasks → evidence)

Each phase is a Parallel Work Unit (§11.4.58) with: RED-first test (§11.4.115), source patch, four-layer coverage (§11.4.4(b)), captured evidence dir under `qa-results/vpn_lan/<phase>/<ts>/`, honest-SKIP when bridge down (§11.4.3), independent review before commit (§11.4.142). **No phase claims PASS without a real evidence path** (§11.4.6).

Phase 0 — Bridge scaffold + svord-doctor preflight `[foundation]`

- **T0.1** Author `.env.example` with the §3 contract (tracked, no secrets).
- **T0.2** Author `tests/lib/svord_bridge.sh` — resolves the env vars, validates them, exposes `bridge_up`, `bridge_require` (exit 2 = OPERATOR-BLOCKED per §11.4.68), `bridge_subnet`, `bridge_host`.
- **T0.3** Author `scripts/svord_doctor.sh` — preflight: env resolvable? hooks executable? `HELIX_BRIDGE_HEALTH` exit 0? remote host reachable? Emits a structured verdict + honest SKIP reason when down.
- **T0.4** Companion docs (§11.4.18): `docs/scripts/svord_doctor.md`.
- **T0.5** Gitignore `.env` (verify already covered by §11.4.30 rule).
- **Evidence:** `svord_doctor.sh` run with bridge down ⇒ `SKIP:network_unreachable_external`; with a fake-up stub ⇒ `UP`. RED: doctor must SKIP (not PASS) when `HELIX_BRIDGE_HEALTH` is unset.
- **Autonomous without secrets:** YES (doctor + SKIP path fully testable with a local stub bridge; the live connection is operator-gated).

Phase 1 — L3 routed gateway + SSRF allowlist reconciliation `[security-critical]`

- **T1.1** Add the `HELIX_BRIDGE_SUBNET` allowlist carve-out to the Dante + Squid config (first-match ABOVE the internal-deny).
- **T1.2** Preserve the RFC1918/metadata floor; re-run S1/S3/S4 guards — must stay GREEN.
- **T1.3** Paired §1.1 mutation: an out-of-allowlist RFC1918 target still denies (`block(N)` / `TCP_DENIED`).
- **T1.4** Route-injection: when the bridge is up, ensure the `10.0.0.0/8` route exists via `ppp0` (svord owns the route; helix_proxy only verifies reachability, never re-routes host tables autonomously — §11.4.133 target-safety).
- **Evidence:** access.log `TCP_TUNNEL/HIER_DIRECT` to an allowlisted host (up) vs `TCP_DENIED/HIER_NONE` to a non-allowlisted RFC1918 host; Dante `block(N)` for the denied one. Honest SKIP when bridge down.

Phase 2 — SMB/CIFS/NMB + NFS `[file-shares]`

- **T2.1** SMB/CIFS mount + list + read + write-back-read round-trip against a VPN share (`smbclient` / `mount.cifs`). NMB name-resolution unicast fallback.
- **T2.2** NFS mount + read + write round-trip (`mount -t nfs`).
- **T2.3** Byte-integrity evidence: sha256 of a written file matches on read-back (real data, §11.4.5).

- **Evidence:** `qa-results/vpn_lan/smb/<ts>/roundtrip.evidence` + `nfs/<ts>/`. Honest SKIP when bridge/share absent.
- **Note:** mounts need L3 routing (Phase 1), NOT SOCKS5.

Phase 3 — FTP/FTPS/SFTP + WebDAV `[file-transfer]`

- **T3.1** FTP passive round-trip (route 21 + pinned passive range; PASV must advertise the 10.x addr). FTPS explicit (`AUTH TLS`) + implicit (990).
- **T3.2** SFTP round-trip (22) — the recommended modern path (single connection, routes/tunnels trivially).
- **T3.3** WebDAV via **existing Squid** (PROPFIND `207` + MKCOL `201`) — the easiest, no new component; enable `extension_methods` if Squid is old; ensure the WebDAV origin's TLS port is in `SSL_Ports` for CONNECT.
- **Evidence:** directory-listing content + 207/201 XML bodies; SFTP byte round-trip sha256. Honest SKIP when bridge absent.

Phase 4 — Email (IMAP/IMAPS, SMTP/submission, POP3/POP3S) `[email]`

- **T4.1** IMAPS `LOGIN` + `LIST` mailbox content (993) via `openssl s_client` / `curl imaps://`.
- **T4.2** SMTP submission authenticated send (587/465, `swaks`) — real message accepted (`250`).
- **T4.3** POP3S retrieve (995).
- **T4.4 Negative test (open-relay guard):** an unauthenticated relay to an external domain MUST fail — proves helix_proxy is not an open relay (§4.3).
- **Evidence:** mailbox LIST content + `250` accepted + the negative relay-refused proof. Honest SKIP when bridge/mail-server absent.

Phase 5 — Discovery reflector (mDNS / SSDP / WS-Discovery / DNS-SD) `[discovery]`

- **T5.1** Design the remote-side reflector (Avahi `enable-reflector=yes` for mDNS; smcroute/SSDP relay for 1900) — **containerized via the containers submodule** (§11.4.76), deployed on the remote-subnet side. **Operator-gated** (remote-host change ⇒ §11.4.122 interactive question BEFORE any remote deployment).
- **T5.2** Client-side enumeration proof: `avahi-browse -at` / `gssdp-discover` surfaces a remote-site service through the reflector; the discovered LOCATION/SRV then reachable via a routed unicast probe.

- **Evidence:** reflected service enumerated on the client side + unicast follow-up reachable. Honest SKIP when reflector not deployed (operator-gated).

Phase 6 — Chromecast / DIAL casting `[casting]`

- **T6.1** Discovery via the Phase-5 reflector (`_googlecast._tcp` mDNS + DIAL SSDP `M-SEARCH`).
- **T6.2** Control routed: `GET http://<ip>:8008/setup/eureka_info` (device name JSON) + CASTV2 status (8009 TLS) via `catt`/`go-chromecast`.
- **T6.3** Liveness (§11.4.107): a cast **status transition** observed, not a single frame.
- **Evidence:** eureka_info JSON `name` read + cast status transition. Honest SKIP when no cast device / reflector.

Phase 7 — ADB over VPN (access, debug, connect, flash) `[device-bridge]`

- **T7.1** Routed TCP 5555 `adb connect 10.x:5555` — no proxy hop (recon 4).
- **T7.2** Central adb server model (one adb server, multiple remote devices).
- **T7.3** Debug: `adb shell` / `logcat` / `pull` / `push` round-trip evidence.
- **T7.4 Flash:** `fastboot` is USB-level → route via **`usbip`** (USB-over-IP) from a remote host with the device attached; network `fastboot` is honestly USB-bound (recon 4 FACT). Honest boundary documented (§11.4.6).
- **Evidence:** `adb devices` shows the remote serial; `adb shell getprop` content; a `usbip` fastboot smoke (or honest SKIP + the documented USB-bound boundary). Operator-gated for any flash on a real device (§11.4.133 target-hardware-safety + §11.4.122).

Phase 8 — Miracast verdict (§11.4.112 structurally-impossible) `[honest-boundary]`

- **T8.1** Record the **Won't-fix: structurally-impossible** classification with cited Wi-Fi-Alliance evidence (Miracast = Wi-Fi-Direct / L2, no IP hop for an L3 VPN to route — recon 5 FACT).
- **T8.2** Offer the routable alternative: **Google Cast / DIAL** (Phase 6) as the drop-in "cast to remote display" capability, OR a remote-site Miracast receiver driven by something the VPN can reach.
- **Evidence:** the cited spec text is the artifact; the positive capability lives in Phase 6. NO fake traversal test.

Phase 9 — Full documentation `[docs]`

- User guide, admin manual, per-protocol tutorials, FAQ, architecture diagrams (the §2 routing map + a sequence diagram per protocol class), SQL definitions (if a discovery/inventory DB lands), the bridge-setup guide. All §11.4.65 four-format-adjacent (HTML+PDF), §11.4.168 visually validated (no raw diagram source leaking into PDFs), §11.4.44 revision headers.

Phase 10 — Containerization (containers submodule §11.4.76) `[infra]`

- The reflector, the adb server, and any helper services boot **on-demand via `submodules/containers` `pkg/boot`/`pkg/compose`/`pkg/health`** (rootless Podman §11.4.161) — no ad-hoc podman. Missing capability ⇒ extend the submodule upstream (§11.4.74), never worked around.

Phase 11 — Full test coverage (§11.4.169) + Challenges + HelixQA `[verification]`

- Every phase's protocol gets: unit (parsers/config), integration (real service via bridge), e2e (full round-trip), full-automation (re-runnable, no manual step §11.4.98), security (SSRF allowlist teeth + open-relay guard), stress+chaos (§11.4.85), a **Challenge** entry (challenges submodule), and a **HelixQA bank** case (`tools/helixqa/banks/`) driven by an autonomous session. Bridge-down ⇒ honest SKIP across the board (§11.4.3), never a fake PASS.

---

6. Test-evidence strategy (autonomous, anti-bluff)

- **Bridge-up path** (operator supplies live svord connection + secrets): full round-trip evidence per protocol — real bytes, real mailbox content, real device serials, sha256 integrity.
- **Bridge-down path** (default autonomous, no secrets): `svord-doctor` reports down ⇒ every protocol test **SKIPs with `network_unreachable_external`** (§11.4.3 / §11.4.68 exit-2 OPERATOR-BLOCKED), the harness surfaces the SKIP honestly — **never** a metadata-only or absence-of-error PASS.
- **Local-stub path** (autonomous, no live VPN): the config/parser/allowlist logic (Phase 1 SSRF teeth, Phase 0 doctor, WebDAV method handling) is provable against a **local** loopback service, so the security-critical logic is GREEN without the live VPN.

- Every PASS via `ab_pass_with_evidence` citing a real artifact (§11.4.69); every analyzer self-validated golden-good/golden-bad (§11.4.107(10)).

---

7. Risk register / danger zones (§11.4.172)

Risk	Severity	Mitigation
SSRF allowlist widens egress → internal-network exposure	HIGH	§4 reconciliation: RFC1918 floor kept, narrow carve-out only, paired §1.1 teeth, S1/S3/S4 stay GREEN
helix_proxy becomes an open mail relay	HIGH	§4.3 open-relay guard: authenticated submission only, never anonymous CONNECT-:25, negative test
Modifying svord_toolkit / remote hosts	HIGH	§11.4.122 — interactive question with options BEFORE any change; bridge is invocation-only
Flashing a real device bricks it	HIGH	§11.4.133 target-hardware-safety + operator-gated; usbip flash is operator-authorized only
Multicast reflector deployed on remote host without asking	MEDIUM	Phase 5 operator-gated (§11.4.122); local-stub proof first
Miracast mis-sold as "supported"	MEDIUM	§11.4.112 honest verdict; Cast offered as the real alternative
Bridge-down tests fake-PASS	HIGH	§11.4.3 honest SKIP enforced by svord-doctor; §11.4.69 no fail-open-skip
Secrets leak into git / .env template	HIGH	§11.4.10 — `.env` gitignored, `.env.example` names/paths only, pre-store leak audit §11.4.10.A

---

8. Autonomous vs operator-gated split (§11.4.101 / §11.4.126)

Autonomous now (no operator input): Phase 0 (bridge scaffold + doctor + SKIP path), Phase 1 SSRF-allowlist logic + teeth against a local stub, Phase 8 Miracast verdict, Phase 9 docs, Phase 11 test scaffolding with honest-SKIP, all parser/config unit tests.

Operator-gated (parked, §11.4.21): live svord connection (secrets + Mullvad + root sudoers), any remote-host reflector deployment (§11.4.122), any real-device flash (§11.4.133), the bridge-up round-trip evidence per protocol.

The autonomous slate is dispatched in parallel per §11.4.103; operator-gated items are surfaced via §11.4.66 interactive options when reached, never silently blocking the loop.

---

9. Sequencing

```
Phase 0 (scaffold) ─┬─> Phase 1 (routing+SSRF) ─┬─> Phase 2 (SMB/
NFS)
                    │                           ├─> Phase 3 (FTP/
WebDAV)
                    │                           ├─> Phase 4
(email)
                    │                           └─> Phase 7 (ADB)
                    ├─> Phase 5 (reflector) ────> Phase 6 (Cast)
                    ├─> Phase 8 (Miracast verdict — independent)
                    └─> Phase 9/10/11 (docs/containers/tests —
continuous)
```

Phase 0 + Phase 1 are the critical path (everything unicast depends on the routed gateway + reconciled SSRF). Phase 5 (reflector) is the critical path for Phase 6 (Cast). Phase 8 is independent. Docs/containers/tests run continuously alongside.

---

10. Honest boundary (§11.4.6)

This plan reflects the recon-derived architecture as of 2026-07-01. It guarantees a **correct, security-reconciled, evidence-driven** design — it does NOT guarantee live protocol success until the operator supplies the svord bridge connection (secrets + Mullvad + root sudoers). Until then the autonomous slate proves the logic against local stubs + honest-SKIPs the live paths (§11.4.3). Miracast is classified structurally-impossible (§11.4.112) with

cited evidence — a durable but not eternal verdict (reopen requires NEW evidence the platform constraint changed, §11.4.34/§11.4.7). No phase is "done" until its runtime signature verifies with captured evidence (§11.4.108) and it crosses independent review (§11.4.142) + the §11.4.169 test-type matrix.

---

11. Deep-research citations (§11.4.150 — multi-angle, cited)

Recon streams (5) completed 2026-07-01, cited authoritative sources (access date 2026-07-01):

- **SMB/NFS/mDNS + routing gap:** L3 route + scoped allowlist; multicast needs remote reflector (recon 1–3).
- **ADB over VPN:** routed 5555 + central adb server; `usbip` for USB-level fastboot; network fastboot honestly USB-bound (recon 4).
- **FTP/WebDAV/email/Cast/Miracast (recon 5, full URLs in the recon report):** FTP active/passive + FTPS ALG-blindness (slacksite, exavault, WinSCP, Wikipedia/FTPS); SFTP single-channel (sftptogo, SolarWinds); WebDAV = HTTP RFC 4918 + Squid `extension_methods`/`SSL_Ports` (rfc-editor, squid-cache wiki); email ports + STARTTLS + RFC 8314 + open-relay (smtpedia, mailgun, fastmail, threatmon); Chromecast CASTV2 8008/8009 + mDNS discovery (CR-Cast wiki, kiljan.org); DIAL = SSDP 1900 + HTTP (DIAL spec 1.6.4, williamboles, Amazon); **Miracast = Wi-Fi-Direct/L2 structurally-impossible** (Wi-Fi Alliance Miracast Technical Overview, Wikipedia/Miracast, devopedia); multicast reflectors (spinetix, twisteroidambassador, mikrotik).

Full URL list preserved in the recon-5 report + to be copied into the per-phase closure footers as each phase lands (§11.4.150 `Deep-research <date>: <urls>`).